HO CHI MINH CITY UNIVERSITY OF SCIENCE
Faculty of Information Technology

REPORT

Group Project Report:

Sorting Algorithms

Course: Data Structures & Algorithms

Authors:

Minh Nhat Nguyen (25127446)

Anh Thu Huu Le (25127512)

Huy Khanh Nguyen (25127375)

Nhat Ha Huynh (25127320)

Instructors:

M.S. Thao Nhi Thi Tran

M.S. Quan Hoang Tran

Contents

1 Group Information 3

2 Algorithm Presentation 4

2.1 Selection Sort 4

2.1.1 Core Concepts 4

2.1.2 Step-by-step Explanations 4

2.1.3 Complexity Analysis 5

2.1.4 Variants and Optimizations 6

2.2 Insertion Sort 6

2.2.1 Core Concepts 6

2.2.2 Step-by-step Explanations 6

2.2.3 Complexity Analysis 7

2.2.4 Variants and Optimizations 7

2.3 Shell Sort 7

2.3.1 Core Concepts 7

2.3.2 Step-by-step Explanations 8

2.3.3 Complexity Analysis 8

2.3.4 Variants and Optimizations 9

2.4 Bubble Sort 9

2.4.1 Core Concepts 9

2.4.2 Step-by-step Explanations 9

2.4.3 Complexity Analysis 9

2.4.4 Variants and Optimizations 9

2.5 Heap Sort 10

2.5.1 Core Concepts 10

2.5.2 Step-by-step Explanations 10

2.5.3 Complexity Analysis 15

2.5.4 Variants and Optimizations 16

2.6 Merge Sort 16

2.6.1 Core Concepts 16

2.6.2 Step-by-step Explanations 16

2.6.3 Complexity Analysis 18

2.6.4 Variants and Optimizations 18

2.7 Quick Sort 18

2.7.1 Core Concept 18

2.7.2 Step-by-step Explanations quick sort 19

2.7.3 Complexity Evaluation 20
2.8 Radix Sort 21
2.8.1 Core Concepts 21
2.8.2 Step-by-step Explanations 21
2.8.3 Complexity Analysis 21
2.8.4 Variants and Optimizations 22
2.9 Counting Sort 22
2.9.1 Core Concepts 22
2.9.2 Step-by-step Explanations 22
2.9.3 Complexity Analysis 23
2.9.4 Variants and Optimizations 24
2.10 Binary Insertion Sort 24
2.10.1 Core Concepts 24
2.10.2 Step-by-step Explanations 25
2.10.3 Complexity Analysis 26
2.10.4 Variants and Optimizations 26
2.11 Shaker Sort 27
2.11.1 Core Concepts 27
2.11.2 Step-by-step Explanations 27
2.11.3 Complexity Analysis 28
2.11.4 Variants and Optimizations 29
2.12 Flash Sort 29
2.12.1 Core Concepts 29
2.12.2 Step-by-step Explanations 29
2.12.3 Complexity Analysis 30
2.12.4 Variants and Optimizations 32
3 Experimental results and comments 33
3.1 Table 33
3.1.1 Data order: Randomized 33
3.1.2 Data order: Sorted 34
3.1.3 Data order: Reversed 34
3.1.4 Data order: Nearly Sorted 35
3.2 Visualizing 36
3.2.1 Line Graphs 36
3.2.2 Bar Charts 40
3.2.3 Overall Comments 44
4 Project organization and Programming notes 45
4.0.1 Project organization 45
4.0.2 Programming notes 46
5 References 48
6 Acknowledgements 50

1. Group Information

Student Name	ID	Primary Responsibilities	Contribution %
Minh Nhat Nguyen	25127446	Selection Sort, Heap Sort, Counting Sort, main.cpp (CLI/dispatcher), command-line handling, file integration, report, bar charts	27
Anh Thu Huu Le	25127512	Quick Sort, Merge Sort, Shell Sort, report writing for assigned algorithms	25
Huy Khanh Nguyen	25127375	Bubble Sort, benchmark/timer function, line graph, report formatting, results section and charts	23
Nhat Ha Huynh	25127320	Insertion Sort, Radix Sort, Binary Insertion Sort, Shaker Sort, Flash Sort, report writing for assigned algorithms	25
		Total	100%

2. Algorithm Presentation

2.1. Selection Sort

2.1.1. Core Concepts

Selection Sort is a sorting algorithm based on the brute-force approach. The algorithm begins by searching for the smallest element in the array and swapping it with the first element. Next, it finds the smallest element among the remaining $n-1$ elements and swaps it with the second element. This process is repeated, each time selecting the smallest element from the unsorted portion of the array and placing it in its correct position, until the entire array becomes sorted.

Selection Sort is well known for its simplicity and ease of implementation. However, its main disadvantage is its time complexity, which remains $O(n^2)$ even in the best case, making it inefficient for large datasets.

2.1.2. Step-by-step Explanations

We have an array:

$$8 \quad 9 \quad 1 \quad 12 \quad -10$$

First, find the smallest value in the array.

$$8 \quad 9 \quad 1 \quad 12 \quad -10$$

Swap it with the first element.

$$-10 \quad 9 \quad 1 \quad 12 \quad 8$$

Now repeat the same process for the remaining unsorted subarray.

$$-10 \quad 9 \quad 1 \quad 12 \quad 8$$

Swap with the second position.

$$-10 \quad 1 \quad 9 \quad 12 \quad 8$$

Find the smallest element in the remaining subarray.

$$-10 \quad 1 \quad 9 \quad 12 \quad 8$$

Swap with the third position.

$$-10 \quad 1 \quad 8 \quad 12 \quad 9$$

Finally, find the smallest element in the remaining subarray.

$$-10 \quad 1 \quad 8 \quad 12 \quad 9$$

Swap with the fourth position.

$$-10 \quad 1 \quad 8 \quad 9 \quad 12$$

Now the array is sorted.

2.1.3. Complexity Analysis

Selection Sort always scans the remaining unsorted part of the array to find the minimum element.

In the first pass, the algorithm performs $n-1$ comparisons, in the second pass it performs $n-2$ comparisons, and so on, until the last pass performs 1 comparison.

Therefore, the total number of comparisons is:

$$\sum_{i=1}^{n-1}(n-i)$$

This summation can be rewritten as:

$$\sum_{i=1}^{n-1}(n-i) = (n-1) + (n-2) + \cdots + 1$$

Hence,

$$\sum_{i=1}^{n-1}(n-i) = \frac{n(n-1)}{2}$$

Therefore, the time complexity of Selection Sort is $O(n^2)$ in the best case, average case, and worst case, because the algorithm always performs the same sequence of comparisons regardless of the initial order of the input array.

- Best case: $O(n^2)$
- Average case: $O(n^2)$
- Worst case: $O(n^2)$
- Space complexity: $O(1)$

The space complexity is $O(1)$ because Selection Sort is an in-place sorting algorithm and only uses a constant amount of extra memory for temporary variables.

2.1.4. Variants and Optimizations

A common variant of Selection Sort is **stable selection sort**, which preserves the relative order of equal elements by inserting the minimum element into its correct position instead of swapping.

Another variant is **bidirectional selection sort**, which finds both the minimum and maximum elements in each pass and places them at the beginning and the end of the unsorted subarray.

However, these improvements do not change the overall time complexity, which remains $O(n^2)$. Therefore, Selection Sort is mainly suitable for small datasets.

2.2. Insertion Sort

2.2.1. Core Concepts

Insertion sort works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list.

2.2.2. Step-by-step Explanations

For example, consider the input array:

$$A = \{7, 3, 5, 2, 6\}$$

Insertion Sort assumes that the left part of the array is already sorted and inserts each subsequent element into its correct position. The number highlighted in red represents the key, while the underline indicates the sorted part of the array.

Step 1: The first key is at the second array

$$\underline{7} \quad \color{red}{3} \quad 5 \quad 2 \quad 6$$

Since $3 < 7$, shift 7 to the right and insert 3 at the beginning.

$$3 \quad 7 \quad 5 \quad 2 \quad 6$$

Step 2: The second key is 5

$$\underline{3 \quad 7} \quad \color{red}{5} \quad 2 \quad 6$$

Since $5 < 7$, shift 7 to the right. Since $5 > 3$, insert 5 after 3.

$$3 \quad 5 \quad 7 \quad 2 \quad 6$$

Step 3: The second key is 2

$$\underline{3 \quad 5 \quad 7} \quad 2 \quad 6$$

Since 2 is smaller than all elements in the sorted part, shift them to the right and insert 2 at the beginning.

$$2 \quad 3 \quad 5 \quad 7 \quad 6$$

Step 4: The second key is 6

$$\underline{2 \quad 3 \quad 5 \quad 7} \quad 6$$

Since $6 < 7$, shift 7 to the right. Since $6 > 5$, insert 6 after 5.

$$2 \quad 3 \quad 5 \quad 6 \quad 7$$

Final sorted array

$$A = \{2, 3, 5, 6, 7\}$$

2.2.3. Complexity Analysis

- Best case: $O(n)$
- Average case: $O(n^2)$
- Worst case: $O(n^2)$
- Space complexity: $O(1)$

2.2.4. Variants and Optimizations

- Binary Insertion Sort: 2.10
- Shell sort: 2.3

2.3. Shell Sort

2.3.1. Core Concepts

Shell Sort, also known as Shell's method, is an in-place comparison sort and an optimization of Insertion Sort. The method starts by sorting pairs of elements far apart from each other, then progressively reducing the gap between elements to be compared. This allows faster movement of elements across the array.

2.3.2. Step-by-step Explanations

We have an initial array:

$$12 \quad 34 \quad 54 \quad 2 \quad 3$$

Choose the initial gap:

$$\text{Gap} = \lfloor 5/2 \rfloor = 2$$

Compare elements that are two positions apart.

$$12 \quad 34 \quad 54 \quad 2 \quad 3$$

Since $12 < 54$, no change is needed.

$$12 \quad 2 \quad 54 \quad 34 \quad 3$$

Next, compare 34 and 2.

$$12 \quad 34 \quad 54 \quad 2 \quad 3$$

Since $34 > 2$, move 34 forward.

$$12 \quad 2 \quad 54 \quad 34 \quad 3$$

Next, compare 54 and 3.

$$12 \quad 2 \quad 54 \quad 34 \quad 3$$

Since $54 > 3$, move 54 forward.

$$12 \quad 2 \quad 3 \quad 34 \quad 54$$

Reduce the gap to 1 and perform a normal insertion sort.

$$2 \quad 3 \quad 12 \quad 34 \quad 54$$

Now the array is sorted.

2.3.3. Complexity Analysis

Time Complexity:

- Best Case: $O(n \log n)$, when the array is already nearly sorted (depends on gap sequence).
- Average Case: Between $O(n^{1.25})$ and $O(n^{1.5})$, depending on the chosen gap sequence.
- Worst Case: $O(n^2)$, with simple gap sequences like $n/2, n/4, \ldots, 1$.

Space complexity: $O(1)$, as it sorts in-place.

2.3.4. Variants and Optimizations

Gap Sequence Selection: The performance is highly sensitive to the gap sequence. Common optimized sequences include:

- **Knuth's Sequence:** $h = 3h + 1$ (e.g., $1, 4, 13, 40, \ldots$).
- **Sedgewick's Sequence:** $9 \cdot 4^i - 9 \cdot 2^i + 1$, which merges various increments to achieve $O(n^{4/3})$ performance.

2.4. Bubble Sort

2.4.1. Core Concepts

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping adjacent elements if they are in the wrong order. This algorithm is not efficient for large data sets as its average and worst-case time complexity are quite high.

2.4.2. Step-by-step Explanations

- In each pass, the algorithm scans the array from the end $(n - 1)$ down to position $i + 1$.
- During the scan, it compares adjacent elements $a[j]$ and $a[j - 1]$. If $a[j]$ is smaller than $a[j - 1]$, they are swapped.
- By repeatedly comparing and swapping adjacent elements in this way, smaller elements gradually move toward the beginning of the array.
- After the first pass, the smallest element moves to the first position. After the second pass, the second smallest element moves to the second position, and so on.
- In general, after k passes, the k smallest elements are placed in the first k positions of the array.

2.4.3. Complexity Analysis

- Time Complexity: $O(n^2)$
- Auxiliary Space: $O(1)$

2.4.4. Variants and Optimizations

Variants:

- Set `swapped = false` at the start of each pass.
- If a swap occurs, set `swapped = true`.
- If `swapped == false` after a pass, stop because the array is already sorted.

Optimizations: Using a boolean `swapped` variable helps detect whether any swap happened during a pass. If no elements are swapped, the algorithm can terminate early. This improves performance for sorted or nearly sorted arrays.

2.5. Heap Sort

2.5.1. Core Concepts

Heap Sort is a comparison-based sorting algorithm that uses the **Binary Heap** data structure.

A **heap** is a specialized complete binary tree that satisfies the heap property. Because it is a complete binary tree, all levels are fully filled except possibly the last level, and the last level is filled from left to right. In practice, a heap is usually stored in an array.

There are two common types of heaps:

- **Max Heap:** the value of each parent node is greater than or equal to the values of its children. Therefore, the largest element is always at the root.
- **Min Heap:** the value of each parent node is smaller than or equal to the values of its children. Therefore, the smallest element is always at the root.

Heap Sort for ascending order usually uses a **Max Heap**. First, the array is transformed into a Max Heap so that the largest element is placed at the root. Then, the root is swapped with the last element of the heap, reducing the heap size by one. After that, the heap property is restored. This process is repeated until all elements are sorted.

Conceptually, Heap Sort is similar to Selection Sort because it repeatedly places the largest element into its correct final position. However, instead of scanning the entire unsorted part to find the maximum element, Heap Sort uses the heap structure to perform this operation more efficiently.

Since building the heap takes $O(n)$ time and each of the n extractions requires $O(\log n)$ time, the overall time complexity of Heap Sort is $O(n \log n)$.

2.5.2. Step-by-step Explanations

We have the array:

$$8 \quad 9 \quad 1 \quad 12 \quad -10$$

For illustration, red represents the parent node and blue represents its child nodes.

To sort the array in ascending order using Heap Sort, we first build a **Max Heap**.

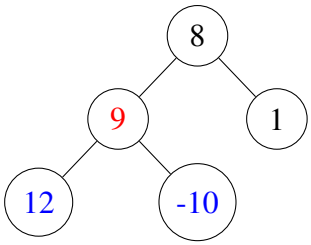
The last non-leaf node is:

$$i = \left\lfloor \frac{n}{2} \right\rfloor - 1$$

Since $n = 5$, we start with $i = 1$.

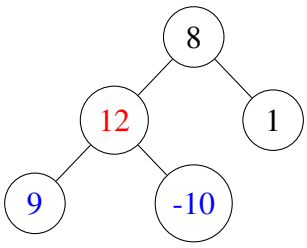
Step 1: Heapify at $i = 1$

$$8 \quad 9 \quad 1 \quad 12 \quad -10$$



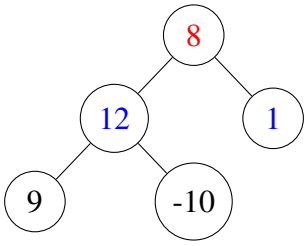
Among the parent 9 and its children 12 and -10, the largest value is 12. So we swap 9 and 12.

$$8 \quad 12 \quad 1 \quad 9 \quad -10$$



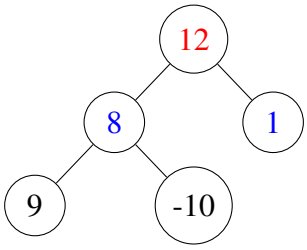
Step 2: Heapify at $i = 0$

$$8 \quad 12 \quad 1 \quad 9 \quad -10$$



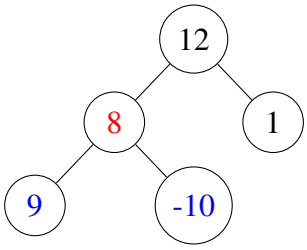
Among the parent 8 and its children 12 and 1, the largest value is 12. So we swap 8 and 12.

$$12 \quad 8 \quad 1 \quad 9 \quad -10$$



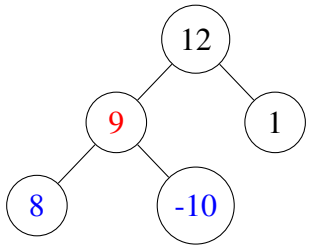
Now we continue heapifying the affected subtree.

$$12 \quad 8 \quad 1 \quad 9 \quad -10$$



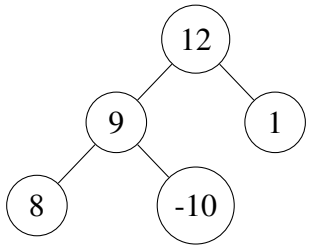
Since $9 > 8$, we swap them.

$$12 \quad 9 \quad 1 \quad 8 \quad -10$$



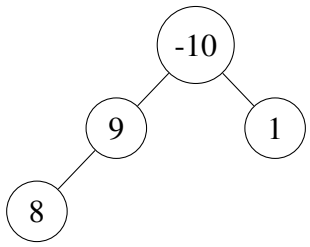
Now the array is a valid **Max Heap**:

$$12 \quad 9 \quad 1 \quad 8 \quad -10$$



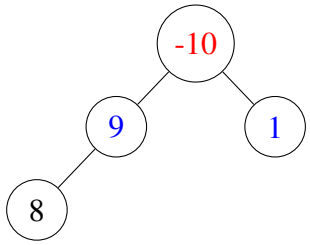
Step 3: Swap the root with the last element

$$-10 \quad 9 \quad 1 \quad 8 \quad \boxed{12}$$



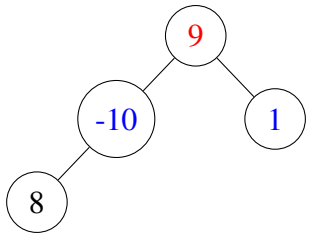
Now reduce the heap size to 4 and heapify the root again.

$$-10 \quad 9 \quad 1 \quad 8 \quad \boxed{12}$$



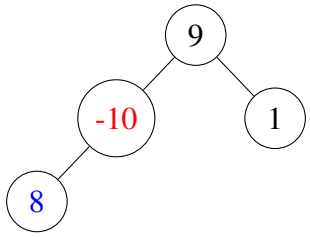
The largest child is 9, so we swap:

$$9 \quad -10 \quad 1 \quad 8 \quad \boxed{12}$$



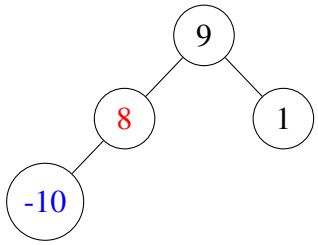
Heapify the affected subtree:

$$9 \quad -10 \quad 1 \quad 8 \quad \boxed{12}$$



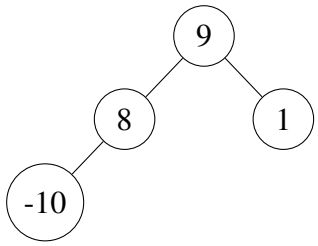
Since $8 > -10$, we swap them:

$$9 \quad 8 \quad 1 \quad -10 \quad \boxed{12}$$



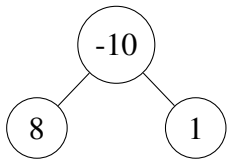
So after heapifying, we get:

$$9 \quad 8 \quad 1 \quad -10 \quad \boxed{12}$$



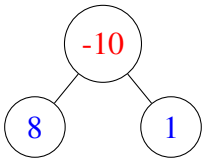
Step 4: Swap the root with the last element in the heap

$$-10 \quad 8 \quad 1 \quad \boxed{9} \quad \boxed{12}$$



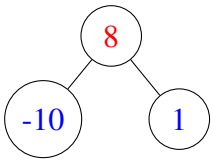
Reduce the heap size to 3 and heapify again.

$$-10 \quad 8 \quad 1 \quad \boxed{9} \quad \boxed{12}$$



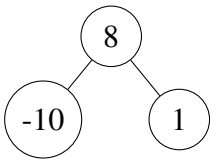
The largest child is 8, so we swap:

$$8 \quad -10 \quad 1 \quad \boxed{9} \quad \boxed{12}$$



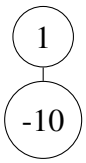
Now the heap becomes:

$$8 \quad -10 \quad 1 \quad \boxed{9} \quad \boxed{12}$$



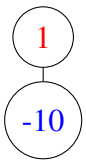
Step 5: Swap the root with the last element in the heap

$$1 \quad -10 \quad \boxed{8} \quad \boxed{9} \quad \boxed{12}$$



Reduce the heap size to 2 and heapify again.

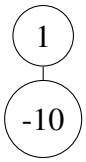
$$1 \quad -10 \quad \boxed{8} \quad \boxed{9} \quad \boxed{12}$$



Since $1 > -10$, no swap is needed.

So the heap remains:

$$1 \quad -10 \quad \boxed{8} \quad \boxed{9} \quad \boxed{12}$$



Step 6: Swap the root with the last element in the heap

$$-10 \quad \boxed{1} \quad \boxed{8} \quad \boxed{9} \quad \boxed{12}$$

Now the heap size is 1, so the algorithm stops.

Thus, the final sorted array is:

$$-10 \quad 1 \quad 8 \quad 9 \quad 12$$

2.5.3. Complexity Analysis

Heap Sort works in two phases: building a Max Heap and repeatedly extracting the maximum element.

After the heap is built, the algorithm performs $n-1$ extractions. In each extraction, the root is swapped with the last element of the heap and the heap size decreases by one. To restore the heap property, the heapify operation is applied.

The height of a binary heap with i elements is $\lfloor \log_2 i \rfloor$. Therefore, the number of comparisons required for each heapify operation is proportional to the height of the heap.

Thus, the total number of comparisons can be expressed as

$$\sum_{i=1}^{n-1} \lfloor \log_2 i \rfloor$$

Using the approximation $\log_2 i \approx \log_2 n$, we obtain

$$\sum_{i=1}^{n-1} \log_2 i$$

This summation is equal to

$$\log_2(1 \cdot 2 \cdot 3 \cdots (n-1))$$

$$= \log_2((n-1)!)$$

Using Stirling's approximation,

$$\log_2((n-1)!) \approx (n-1)\log_2(n-1) - (n-1)$$

Therefore, the total number of comparisons grows proportionally to

$$n \log_2 n$$

- Best case: $n \log n$
- Average case: $n \log n$
- Worst case: $n \log n$
- Space complexity: 1

The space complexity is constant because Heap Sort is an in-place sorting algorithm and only requires a few auxiliary variables.

2.5.4. Variants and Optimizations

One important optimization in Heap Sort is the bottom-up heap construction method, also known as Floyd's algorithm. Instead of inserting elements one by one into the heap, the algorithm builds the heap by calling heapify starting from the last non-leaf node. This reduces the heap construction cost from $n \log n$ to n.

Another common improvement is optimizing the heapify process by reducing unnecessary swaps. Instead of repeatedly swapping elements, some implementations move elements down the heap and perform the final placement only once. This reduces memory operations and improves practical performance.

However, these optimizations do not change the overall time complexity of Heap Sort, which remains $n \log n$ in all cases.

2.6. Merge Sort

2.6.1. Core Concepts

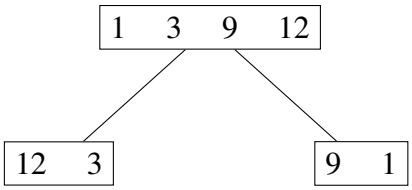
Based on a divide-and-conquer algorithmic strategy, merge sort is a popular sorting algorithm known for its efficiency and stability. It works by recursively splitting the list into halves, sorting each half separately, and then merging them into one sorted array. It is a stable sorting algorithm, meaning that it preserves the relative order of equal elements.

2.6.2. Step-by-step Explanations

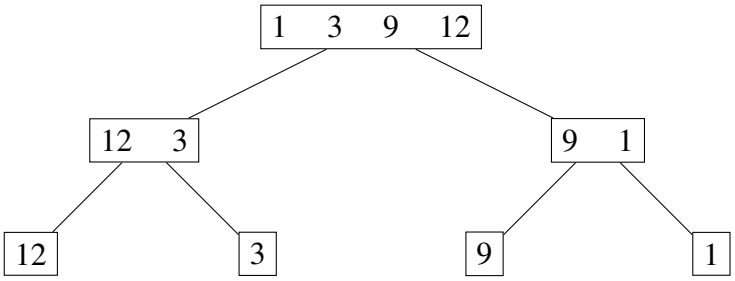
We have an array:

$$12 \quad 3 \quad 9 \quad 1$$

First, split the array into two halves.



Continue splitting until each sub-array contains only one element.



Now compare and merge the elements.

$$[12] \quad [3] \rightarrow [3, 12]$$

$$[9] \quad [1] \rightarrow [1, 9]$$

$$[3 \quad 12] \quad [1 \quad 9]$$

Compare elements step by step:

$$1 < 3 \Rightarrow [1]$$

$$3 < 9 \Rightarrow [1 \quad 3]$$

$$9 < 12 \Rightarrow [1 \quad 3 \quad 9]$$

Remaining element:

$$[1 \quad 3 \quad 9 \quad 12]$$

Final Sorted Array:

1	3	9	12

2.6.3. Complexity Analysis

Merge Sort uses the divide-and-conquer approach. The array is repeatedly divided into two halves until each sub-array contains only one element. After that, the sub-arrays are merged while comparing elements.

At each level of recursion, the merging process scans all elements once. Therefore, the number of comparisons per level is approximately n.

The height of the recursion tree is $\log_2 n$ because the array size is halved at each step.

Thus, the total number of comparisons can be expressed as:

Therefore, the total number of comparisons is:

$$\sum_{i=1}^{\log_2 n} n = n \sum_{i=1}^{\log_2 n} = n \log_2 n$$

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- Worst case: $O(n \log n)$
- Space complexity: $O(n)$

Merge Sort requires additional memory to store temporary sub-arrays during the merging process.

2.6.4. Variants and Optimizations

Small Sub-array Optimization: Transition to Insertion Sort when the sub-array size is small, to reduce recursive overhead.

Merge Sort for Linked Lists:

- No extra temporary arrays.
- Sequential access.
- Stable $O(n \log n)$ time complexity across all cases.

2.7. Quick Sort

2.7.1. Core Concept

QuickSort is a sorting algorithm based on the *Divide and Conquer* strategy. It selects an element from the array as a *pivot* and partitions the remaining elements into two sub-arrays: those less than the pivot and those greater than the pivot.

2.7.2. Step-by-step Explanations quick sort

We start with the initial array:

$$12 \quad 3 \quad 9 \quad 1 \quad 8$$

Choose the middle element as the pivot.

$$12 \quad 3 \quad \color{red}{9} \quad 1 \quad 8$$

Swap the pivot with the first element.

$$\color{red}{9} \quad 3 \quad 12 \quad 1 \quad 8$$

Now the array is divided into regions:

$$[\color{red}{pivot}] \quad [Unknown]$$

$$\color{red}{9} \quad | \quad 3 \quad 12 \quad 1 \quad 8$$

Compare the first unknown element with the pivot.

$$\color{red}{9} \quad \color{blue}{3} \quad 12 \quad 1 \quad 8$$

Since $3 \leq 9$, move it into region $S1$.

$$\color{red}{9} \quad | \quad 3 \quad | \quad 12 \quad 1 \quad 8$$

Next compare 12 with the pivot. Since $12 > 9$, it remains in the right region (S2).

$$[\color{red}{pivot}] \quad [S1] \quad [S2] \quad [Unknown]$$

$$\color{red}{9} \quad | \quad 3 \quad | \quad 12 \quad | \quad 1 \quad 8$$

Next compare 1 with the pivot. Since $1 \leq 9$, swap it with the first element of the right region (S2).

$$\begin{array}{cccc} \text{[pivot]} & \text{[S1]} & \text{[S2]} & \text{[Unknown]} \\ \color{red}{9} & | \ 3 \quad 1 & | \ 12 & | \ 8 \end{array}$$

Next compare 8 with the pivot. Since $8 \leq 9$, move it into region $S1$.

$$[\color{red}{pivot}] \qquad [S1] \qquad [S2]$$

$$\color{red}{9} \quad | \quad 3 \quad 1 \quad 8 \quad | \quad 12$$

Finally, place the pivot into its correct position by swapping it with the last element of $S1$.

$$8 \quad 3 \quad 1 \quad 9 \quad 12$$

Now the pivot is in its final position and the array is divided into two subarrays.

$$8 \quad 3 \quad 1 \quad | \quad 9 \quad | \quad 12$$

Quick Sort is then applied recursively to the left subarray.

$$1 \quad 3 \quad 8 \quad 9 \quad 12$$

2.7.3. Complexity Evaluation

- **Time Complexity:** The efficiency of Quick Sort depends on the balance of the partitions, which is determined by the pivot selection.
  - **Worst-case:** $O(n^2)$
    This occurs when the pivot selection consistently leads to the most unbalanced partitions possible (e.g., the largest or smallest element is always chosen in an already sorted array). In this case, one sub-array is empty while the other has $n-1$ elements.

    $$C(n) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2} = \Theta(n^2)$$

  - **Average-case:** $O(n \log n)$
    This occurs when the pivot p can be at any position with an equal probability of $1/n$. The average number of comparisons is calculated by the recurrence:

    $$C(n) = \frac{1}{n} \sum_{i=1}^{n} [(n-1) + C(i) + C(n-i-1)]$$

    $$\approx 1.39\, n \log n$$

    Order of growth: $O(n \log n)$
  - **Best-case:** $O(n \log n)$
    This occurs when the pivot always splits the array into two equal, or nearly equal, halves.

    $$C(n) = 2C(n/2) + n - 1, \quad (n > 1)$$

    Order of growth: $O(n \log n)$

- **Space Complexity:**

- **Worst-case:** $O(n)$

  When the pivot selection consistently leads to unbalanced partitioning, the recursion tree becomes skewed, resulting in a stack depth of n levels.

- **Best/Average-case:** $O(\log n)$

  When Quick Sort achieves balanced partitioning, the recursion tree is approximately $\log(n)$ levels deep, requiring proportional stack space.

2.8. Radix Sort

2.8.1. Core Concepts

Radix Sort is a non-comparison-based sorting algorithm that sorts numbers by processing individual digits. Instead of comparing elements directly, Radix Sort groups numbers according to their digits at specific positions (such as units, tens, hundreds, etc.).

The algorithm processes digits one position at a time, usually starting from the least significant digit (LSD) and moving toward the most significant digit (MSD). At each step, a stable sorting algorithm such as Counting Sort is used to sort the numbers based on the current digit.

Radix Sort is particularly efficient when sorting integers with a limited number of digits.

2.8.2. Step-by-step Explanations

1. Find the maximum number in the array to determine the number of digits.

2. Starting from the least significant digit, repeat the following steps while there are still digits to process:

   (a) Use Counting Sort to sort the array according to the current digit (determined by the exponent value).

   (b) Increase the exponent by a factor of 10 to move to the next more significant digit.

3. Repeat the process until the most significant digit has been processed.

   For example, if we have an array A of input [25, 320, 113, 250]. The operation of Radix Sort will look like this:

3	2	0
2	5	0
1	1	3
0	2	5

$\longrightarrow$

1	1	3
3	2	0
0	2	5
2	5	0

$\longrightarrow$

0	2	5
1	1	3
2	5	0
3	2	0

$\longrightarrow$

0	2	5
1	1	3
2	5	0
3	2	0

2.8.3. Complexity Analysis

- Time Complexity: $O(d(n + b))$

where d is the number of digits, n is the number of elements, and b is the base of the number system used.

Radix Sort is often faster than comparison-based sorting algorithms for large datasets with small digit lengths. However, it requires additional memory for digit counting, so it may not be efficient for very small datasets.

- Space Complexity: $O(n + k)$

where n is the number of elements in the input array and k is the base of the number system.

2.8.4. Variants and Optimizations

- **Least Significant Digit (LSD) Radix Sort**: The most common version, which processes digits starting from the least significant digit.
- **Most Significant Digit (MSD) Radix Sort**: Sorting starts from the most significant digit and recursively processes subgroups.
- **Hybrid Approaches**: Radix Sort is sometimes combined with other algorithms such as Insertion Sort for small subarrays to improve efficiency.

2.9. Counting Sort

2.9.1. Core Concepts

Counting Sort is a non-comparison sorting algorithm that works by counting the number of occurrences of each distinct value in the input array.

First, the algorithm finds the minimum and maximum values in the array in order to determine the range of the input data. Then, it creates a counting array of size

$$\text{maxVal} - \text{minVal} + 1$$

Each element in the input array is mapped to an index in the counting array, and its frequency is recorded.

After that, the counting array is transformed into a prefix sum array. This prefix sum indicates how many elements are less than or equal to a given value, which helps determine the correct position of each element in the sorted output.

Finally, the algorithm places each element into its correct position in the output array based on the prefix sums. Since Counting Sort does not compare elements directly, it is efficient when the range of input values is not too large.

2.9.2. Step-by-step Explanations

Consider the input array:

$$4 \quad 2 \quad 2 \quad 8 \quad 3 \quad 3 \quad 1$$

The maximum value is 8, so we create a counting array from 0 to 8.

Counting array after counting occurrences:

Index	0	1	2	3	4	5	6	7	8
Count	0	1	2	2	1	0	0	0	1

Prefix sum array:

Index	0	1	2	3	4	5	6	7	8
Prefix Sum	0	1	3	5	6	6	6	6	7

Now we place each element into the output array according to the prefix sums, scanning the input from right to left:

$$[1\ 2\ 2\ 3\ 3\ 4\ 8]$$

Thus, the final sorted array is:

$$1 \quad 2 \quad 2 \quad 3 \quad 3 \quad 4 \quad 8$$

2.9.3. Complexity Analysis

Counting Sort consists of several linear passes over the input array and the counting array.

First, we scan the input array to find the maximum value:

$$\sum_{i=1}^{n} 1$$

Next, we initialize the counting array of size $k + 1$:

$$\sum_{i=0}^{k} 1$$

Then we count the occurrences of each element:

$$\sum_{i=1}^{n} 1$$

After that, we compute the prefix sums of the counting array:

$$\sum_{i=1}^{k} 1$$

Next, we traverse the input array again to place elements into the output array:

$$\sum_{i=1}^{n} 1$$

Finally, we copy the output array back to the original array:

$$\sum_{i=1}^{n} 1$$

Therefore, the total number of operations is:

$$\sum_{i=1}^{n} 1 + \sum_{i=0}^{k} 1 + \sum_{i=1}^{n} 1 + \sum_{i=1}^{k} 1 + \sum_{i=1}^{n} 1 + \sum_{i=1}^{n} 1$$

$$= n + (k+1) + n + k + n + n$$

$$= 4n + 2k + 1$$

Thus, the running time grows proportionally to

$$n + k$$

- Best case: $n + k$
- Average case: $n + k$
- Worst case: $n + k$
- Space complexity: $n + k$

2.9.4. Variants and Optimizations

A practical optimization for Counting Sort is to use a fixed-size counting array when the maximum value is small and the input range is dense. For example, if all input values lie in the range 0 to 255, the counting array can be declared with size 256 directly. This avoids dynamic memory allocation overhead and simplifies the implementation.

However, this optimization is only effective when the value range is small. If the maximum value is very large or the data is sparse, Counting Sort becomes less efficient in both time and memory usage.

2.10. Binary Insertion Sort

2.10.1. Core Concepts

Binary Insertion Sort find the correct position where the element should be inserted in the sorted part of the array.

This reduces the number of comparisons needed.

However, elements still need to be shifted, so the total running time does not improve much.

2.10.2. Step-by-step Explanations

For example, consider the input array:

$$A = \{9, 3, 7, 2, 8, 5, 1\}$$

Binary Insertion Sort assumes that the left part of the array is already sorted and uses binary search to find the correct position for the key. The number highlighted in red represents the key, while the underline indicates the sorted part of the array.

Step 1: The second element is the first key

$$\underline{9} \quad 3 \quad 7 \quad 2 \quad 8 \quad 5 \quad 1$$

Binary search is applied to the sorted part $\{9\}$. Since $3 < 9$, the correct position of 3 is before 9.

$$3 \quad 9 \quad 7 \quad 2 \quad 8 \quad 5 \quad 1$$

Step 2: The next key is 7

$$\underline{3 \quad 9} \quad 7 \quad 2 \quad 8 \quad 5 \quad 1$$

Binary search is applied to the sorted part $\{3, 9\}$. Since $3 < 7 < 9$, the correct position of 7 is between 3 and 9.

$$3 \quad 7 \quad 9 \quad 2 \quad 8 \quad 5 \quad 1$$

Step 3: The next key is 2

$$\underline{3 \quad 7 \quad 9} \quad 2 \quad 8 \quad 5 \quad 1$$

Binary search is applied to the sorted part $\{3, 7, 9\}$. Since $2 < 3$, the correct position of 2 is at the beginning.

$$2 \quad 3 \quad 7 \quad 9 \quad 8 \quad 5 \quad 1$$

Step 4: The next key is 8

$$\underline{2 \quad 3 \quad 7 \quad 9} \quad 8 \quad 5 \quad 1$$

Binary search is applied to the sorted part $\{2, 3, 7, 9\}$. Since $7 < 8 < 9$, the correct position of 8 is between 7 and 9.

$$2 \quad 3 \quad 7 \quad 8 \quad 9 \quad 5 \quad 1$$

Step 5: The next key is 5

$$\underline{2 \quad 3 \quad 7 \quad 8 \quad 9} \quad 5 \quad 1$$

Binary search is applied to the sorted part $\{2, 3, 7, 8, 9\}$. Since $3 < 5 < 7$, the correct position of 5 is between 3 and 7.

$$2 \quad 3 \quad 5 \quad 7 \quad 8 \quad 9 \quad 1$$

Step 6:The next key is 1

$$\underline{2 \quad 3 \quad 5 \quad 7 \quad 8 \quad 9} \quad 1$$

Binary search is applied to the sorted part $\{2, 3, 5, 7, 8, 9\}$. Since $1 < 2$, the correct position of 1 is at the beginning.

$$1 \quad 2 \quad 3 \quad 5 \quad 7 \quad 8 \quad 9$$

Final sorted array

$$A = \{1, 2, 3, 5, 7, 8, 9\}$$

2.10.3. Complexity Analysis

- Best case: $O(n)$
- Average case: $O(n^2)$
- Worst case: $O(n^2)$
- Space complexity: $O(1)$

2.10.4. Variants and Optimizations

Insertion Sort with sentinel – reduces boundary checks.

Hybrid sorting algorithms - use insertion sort for small arrays.

2.11. Shaker Sort

2.11.1. Core Concepts

Shaker Sort, also known as Cocktail Shaker Sort, is an extension of Bubble Sort. Unlike Bubble Sort, which only moves larger elements toward the end of the array, Shaker Sort works in both directions during each pass.

The algorithm alternates between two phases:

- A forward pass that moves the largest element of the unsorted subarray to the end.
- A backward pass that moves the smallest element of the unsorted subarray to the beginning.

By performing sorting in both directions, Shaker Sort can reduce the number of passes required compared to the standard Bubble Sort when small elements are located near the end of the array.

2.11.2. Step-by-step Explanations

The operation of Shaker Sort can be described as follows:

1. Initialize two boundaries: `left` at the beginning of the array and `right` at the end.
2. Perform a forward pass from `left` to `right`, swapping adjacent elements if they are in the wrong order.
3. After the forward pass, the largest element is placed at position `right`. Decrease `right`.
4. Perform a backward pass from `right` to `left`, swapping adjacent elements if necessary.
5. After the backward pass, the smallest element is placed at position `left`. Increase `left`.
6. Repeat the process until the two boundaries meet or no swaps occur.

   For example, consider the input array:

   $$A = \{10, 5, 7, 2, 5\}$$

   left is at index 0, right is at index n-1. Elements being compared are pointed by arrow.

   Step 1: Initial array

   $$\overset{\downarrow}{10} \quad 5 \quad 7 \quad 2 \quad 5$$

   Starting the first forward pass.

   Step 2: Compare 10 to others

$$5 \quad 7 \quad 2 \quad 5 \quad \overset{\downarrow}{10}$$

Since 10 is larger than the other elements, it is moved toward the end of the array.

Step 3: Decrease right and start the backward pass

$$5 \quad 7 \quad 2 \quad \overset{\downarrow}{5} \quad 10$$

In the backward pass, adjacent elements are swapped from right to left so that smaller elements move to the beginning.

Step 4: Continue the backward pass

$$\overset{\downarrow}{2} \quad 5 \quad 7 \quad 5 \quad 10$$

The smallest elements move toward the beginning of the array.

Step 5: Increase left and continue alternating passes

$$2 \quad \overset{\downarrow}{5} \quad 7 \quad 5 \quad 10$$

The process continues until no swaps occur and the array becomes fully sorted:

$$2 \quad 5 \quad 5 \quad \overset{\downarrow}{7} \quad 10$$
$$2 \quad 5 \quad \overset{\downarrow}{5} \quad 7 \quad 10$$
$$2 \quad \overset{\downarrow}{5} \quad 5 \quad 7 \quad 10$$
$$2 \quad 5 \quad 5 \quad 7 \quad 10$$

Final sorted array:

$$A = \{2, 5, 5, 7, 10\}$$

2.11.3. Complexity Analysis

The time and space complexity of Shaker Sort are similar to Bubble Sort.

- Best Case: $O(n)$ when the array is already sorted.
- Average Case: $O(n^2)$.
- Worst Case: $O(n^2)$ when the array is sorted in reverse order.
- Space Complexity: $O(1)$ since the algorithm sorts the array in-place.

Although Shaker Sort performs bidirectional passes, the improvement compared to Bubble Sort is generally small, and the overall complexity remains quadratic.

2.11.4. Variants and Optimizations

Several optimizations can be applied to improve the efficiency of Shaker Sort:

- **Early termination:** If no swaps occur during a pass, the algorithm stops early because the array is already sorted.
- **Boundary shrinking:** After each pass, the sorted portion of the array increases, reducing the range that needs to be checked.
- **Tracking last swap position:** The algorithm can remember the last swap index to further reduce unnecessary comparisons.

These optimizations help reduce redundant comparisons but do not change the overall time complexity of the algorithm.

2.12. Flash Sort

2.12.1. Core Concepts

Flash Sort is a distribution-based sorting algorithm that is particularly efficient for uniformly distributed data. The main idea of the algorithm is to classify the elements of an array into a number of groups, called classes or buckets, using a mapping formula based on the minimum and maximum values of the array.

After the classification step, the algorithm rearranges the elements so that each element is moved close to its final sorted position. Since the elements are only approximately placed into the correct regions, the array is not completely sorted after this phase. Therefore, a final insertion sort is usually applied to finish the sorting process.

The strength of Flash Sort lies in reducing the amount of disorder in the array before applying a comparison-based sorting method. For data that is nearly uniformly distributed, this makes the algorithm significantly faster than simple quadratic sorting algorithms.

2.12.2. Step-by-step Explanations

The operation of Flash Sort can be described as follows:

1. Find the minimum value, maximum value, and their positions in the array.
2. Choose the number of classes m, which is usually proportional to the size of the array.
3. Use a classification formula to map each element into a class:

$$K(A[i]) = \left\lfloor \frac{m-1}{\max - \min} (A[i] - \min) \right\rfloor$$

   where $K(A[i])$ is the class index of element $A[i]$.
4. Count how many elements belong to each class.

5. Convert the class counts into cumulative counts so that each class corresponds to a segment in the array.
6. Rearrange the elements by moving them into their appropriate class regions. This process is called the flashing phase.
7. After the flashing phase, the array is only partially sorted because elements inside each class may still be unordered.
8. Apply insertion sort to the entire array, or to the nearly sorted result, to complete the sorting process.

For example, consider the input array:

$$A = \{7, 2, 5, 1, 6\}$$

Step 1: Find the minimum value, maximum value, and determine the number of buckets.

$$\min = 1, \quad \max = 7, \quad m = 3$$

m denotes the number of buckets used for classification.

Step 2: Classify elements into buckets

$$\{2, 1\} \quad \{5\} \quad \{7, 6\}$$

Each element is mapped into its corresponding bucket according to its value.

Step 3: Rearrange elements into approximate order

$$2 \quad 1 \quad 5 \quad 7 \quad 6$$

The elements are placed back into the array based on the bucket distribution.

Step 4: Apply insertion sort

$$1 \quad 2 \quad 5 \quad 6 \quad 7$$

A final insertion sort is applied to complete the sorting process.

Step 5: Final sorted array

$$A = \{1, 2, 5, 6, 7\}$$

2.12.3. Complexity Analysis

The time complexity of Flash Sort depends on the distribution of the input data and the cost of the final insertion sort.

- **Classification phase:** Finding the minimum and maximum values takes $O(n)$, and counting the number of elements in each class also takes $O(n)$.
- **Flashing phase:** Moving elements into their appropriate class regions takes approximately $O(n)$.
- **Final insertion sort:** In the average case, when the array is already nearly sorted after flashing, insertion sort performs efficiently.

Therefore, the average time complexity of Flash Sort is approximately:

$$T(n) = O(n) + O(n) + O(n) = O(n)$$

This happens when the input data is uniformly distributed, because the classification formula places elements into classes quite evenly, and only a small amount of local disorder remains after flashing.

In the worst case, however, the data may be distributed very unevenly. In that situation, many elements may fall into the same class, so the flashing phase does not reduce disorder effectively. As a result, the final insertion sort may need to process a highly disordered array, giving the worst-case complexity:

$$T(n) = O(n) + O(n^2) = O(n^2)$$

Thus:

- Best / Average Case: $O(n)$
- Worst Case: $O(n^2)$
- Space Complexity: $O(m)$, where m is the number of classes

If m is chosen proportional to n, then the auxiliary space can also be written as $O(n)$. In practice, a common choice is $m \approx 0.1n$ to $0.2n$, depending on the implementation.

Explanation of the classification formula. The formula

$$K(A[i]) = \left\lfloor \frac{m-1}{\max - \min} (A[i] - \min) \right\rfloor$$

maps each element from the value range $[\min, \max]$ into a class index in the range $[0, m-1]$.

- $A[i] - \min$ shifts the value range so that the minimum value becomes 0.
- Dividing by $\max - \min$ normalizes the value into the interval $[0, 1]$.
- Multiplying by $m-1$ stretches this normalized value into the class index range $[0, m-1]$.

- The floor function ensures that the result is an integer class index.

In other words, the formula estimates where an element should be located based on its relative value between the minimum and maximum elements.

2.12.4. Variants and Optimizations

Several optimizations can improve the performance of Flash Sort:

- **Choosing a suitable number of classes:** A good value of m helps distribute elements more evenly among the classes.
- **Improved class mapping:** For non-uniform data, a better mapping strategy may reduce class imbalance.
- **Hybrid finishing step:** Insertion sort is commonly used at the end because the array is already nearly sorted, but other efficient local sorting methods may also be applied.
- **Handling duplicate values carefully:** When many identical values appear, extra care is needed to avoid poor distribution among classes.

These optimizations do not change the worst-case complexity of the algorithm, but they can significantly improve practical performance, especially for large datasets with approximately uniform distribution.

3. Experimental results and comments

3.1. Table

3.1.1. Data order: Randomized

Data order: Randomized						
Data Size	10,000		30,000		50,000	
Resulting statics	Running time (ms)	Comparision	Running time (ms)	Comparision	Running time (ms)	Comparision
Selection Sort	88.7631	100019998	803.09	900059998	2224.7	2500099998
Insertion Sort	95.724	49912158	841.22	448351376	2352.03	1246699556
Shell Sort	3.5311	395342	5.8549	1339074	11.0353	2606713
Bubble Sort	107.271	49984269	1439.61	449927030	4356.18	1249903369
Heap Sort	1.7816	637787	5.4456	2150478	10.1767	3772406
Merge Sort	1.7528	457674	6.1352	1513133	10.4051	2637153
Quick Sort	0.974	336448	4.1146	1219005	6.4129	1968163
Radix Sort	0.3107	140100	1.055	510125	1.6094	850125
Counting Sort	0.166	19998	0.4058	59998	0.5827	99998
Binary Insertion Sort	36.335	25531032	286.591	226592559	731.729	624712946
Shaker Sort	339.84	100005001	3123.75	900015001	8475.39	2500025001
Flash Sort	0	36695	3.536	110786	8.763	173827

Data order: Randomized						
Data Size	100,000		300,000		500,000	
Resulting statics	Running time (ms)	Comparision	Running time (ms)	Comparision	Running time (ms)	Comparision
Selection Sort	8962.82	10000199998	96366.5	90000599998	288377	250000999998
Insertion Sort	9494.98	5000100020	98497.4	45058506959	289374	125079899360
Shell Sort	20.8099	5797367	73.6821	19697452	118.205	36062322
Bubble Sort	33012.2	4999882104	169903	44999537555	38499000	124999166260
Heap Sort	21.4522	8044456	67.1837	26489194	119.934	45968439
Merge Sort	21.2271	5573966	62.0211	18136485	102.347	31287873
Quick Sort	12.084	4129841	42.2674	15322345	70.6597	28589928
Radix Sort	3.2838	1700125	13.2976	5100125	21.1124	8500125
Counting Sort	0.9014	199998	1.9169	599998	2.2835	999998
Binary Insertion Sort	2865.02	2501191439	80673.7	22501899007	192371	62477690757
Shaker Sort	35497.1	10000050001	322167	90000150001	1.05777e+006	250000250001
Flash Sort	4.767	299998	22.974	899998	23.564	1499998

3.1.2. Data order: Sorted

Data order: Sorted						
Data Size	10,000		30,000		50,000	
Resulting statics	Running time (ms)	Comparision	Running time (ms)	Comparision	Running time (ms)	Comparision
Selection Sort	87.327	100019998	807.63	900059998	2751.02	2500099998
Insertion Sort	0	29998	0	89998	0	149998
Shell Sort	0.3577	240037	5.8549	1339074	2.0312	1400043
Bubble Sort	0.0084	9999	0.0288	29999	0.0375	49999
Heap Sort	1.3957	670329	4.7219	2236648	7.7797	3925351
Merge Sort	1.1025	401834	3.5485	1323962	6.3325	2301434
Quick Sort	0.4383	245004	1.2278	823679	2.6957	1467299
Radix Sort	0.4219	140100	0.9951	510125	1.7173	850125
Counting Sort	0.0624	19998	0.2083	59998	0.3105	99998
Binary Insertion Sort	0.202	380851	0.7127	1281699	1.3002	2253395
Shaker Sort	264.113	100005001	2638.34	900015001	7174.25	2500025001
Flash Sort	0	40997	0	122997	3.476	204997

Data order: Sorted						
Data Size	100,000		300,000		500,000	
Resulting statics	Running time (ms)	Comparision	Running time (ms)	Comparision	Running time (ms)	Comparision
Selection Sort	8986.69	10000199998	95447.8	90000599998	345805	250000999998
Insertion Sort	0	299998	1.77	899998	2.51	1499998
Shell Sort	5.0109	3000045	17.3143	10200053	25.143	17000051
Bubble Sort	0.0851	99999	0.2535	299999	2.4005	499999
Heap Sort	16.5657	8365080	52.7981	27413230	92.7592	47404886
Merge Sort	13.1125	4852874	39.6702	15729866	63.9715	27143914
Quick Sort	4.6996	3134535	16.378	10258288	31.1901	17737935
Radix Sort	3.2877	1700125	14.2035	6000150	24.4879	10000150
Counting Sort	0.5195	199998	1.6496	599998	2.8369	999998
Binary Insertion Sort	3.0756	4806787	66.838	15827139	97.3479	27427139
Shaker Sort	37156.1	10000050001	400857	90000150001	420783	250000250001
Flash Sort	5.124	409997	23.366	1229997	13.2454	2049997

3.1.3. Data order: Reversed

Data order: Reversed						
Data Size	10,000		30,000		50,000	
Resulting statics	Running time (ms)	Comparision	Running time (ms)	Comparision	Running time (ms)	Comparision
Selection Sort	86.5508	100019998	793.159	900059998	2226.31	2500099998
Insertion Sort	355.986	100009999	3018.35	900029999	7902.5	2500049999
Shell Sort	0.6744	302597	2.5401	987035	3.2413	1797323
Bubble Sort	106.609	49995000	999.28	449985000	2767.26	1249975000
Heap Sort	1.4882	606771	4.7256	2063324	7.5545	3612724
Merge Sort	1.3079	406234	3.9123	1332186	6.4534	2320874
Quick Sort	0.5026	260834	1.9771	881279	3.3427	1548692
Radix Sort	0.4186	140100	0.9772	510125	1.7318	850125
Counting Sort	0.0697	19998	0.1707	59998	0.2856	99998
Binary Insertion Sort	67.8953	50358178	579.693	451217592	1422.27	1252130139
Shaker Sort	677.458	100005001	5912.02	900015001	33313.3	2500025001
Flash Sort	4.937	37500	9.798	112500	7.736	187500

Data order: Reversed						
Data Size	100,000		300,000		500,000	
Resulting statics	Running time (ms)	Comparision	Running time (ms)	Comparision	Running time (ms)	Comparision
Selection Sort	9039.84	10000199998	89228.6	90000599998	247913	250000999998
Insertion Sort	35207.2	100000999999	244152	900002999999	494919	2500004999999
Shell Sort	7.7557	3844605	31.152	12700933	36.8529	21428803
Bubble Sort	11364.7	4999950000	101214	44999850000	1411060	124999750000
Heap Sort	16.7411	7718943	52.4982	25569379	91.1224	44483348
Merge Sort	13.2079	4891754	38.0656	15848682	63.0054	27234634
Quick Sort	5.5142	3298643	21.2394	10784604	30.8176	18763432
Radix Sort	3.8453	1700125	15.041	6000150	24.0997	10000150
Counting Sort	0.5822	199998	1.6344	599998	2.6319	999998
Binary Insertion Sort	19862.2	5004560230	177410	45015170409	491806	125026390764
Shaker Sort	38515.3	10000050001	380386	90000150001	1.10424e+06	250000250001
Flash Sort	3.681	375000	11.929	1125000	17.136	1875000

3.1.4. Data order: Nearly Sorted

Data order: Nearly Sorted						
Data Size	10,000		30,000		50,000	
Resulting statics	Running time (ms)	Comparision	Running time (ms)	Comparision	Running time (ms)	Comparision
Selection Sort	87.8341	100019998	785.936	900059998	2767.66	2500099998
Insertion Sort	0	177182	1.549	570946	2.177	711638
Shell Sort	0.5232	267635	2.1537	864343	2.5338	1480151
Bubble Sort	37.603	49872240	346.449	429776097	716.277	1033145100
Heap Sort	1.4928	669777	5.1231	2236495	8.0323	3925348
Merge Sort	2.956	415114	3.8019	1379121	6.7272	2345550
Quick Sort	0.7328	261733	1.5083	872916	3.4009	1529513
Radix Sort	0.4374	140100	1.1017	510125	1.6166	850125
Counting Sort	0.0629	19998	0.1527	59998	0.253	99998
Binary Insertion Sort	0.2964	420728	0.8201	1394799	1.5128	2455543
Shaker Sort	172.979	100005001	1558.76	900015001	4124.19	2500025001
Flash Sort	1.004	40985	1.99	122989	2.004	204987

Data order: Nearly Sorted						
Data Size	100,000		300,000		500,000	
Resulting statics	Running time (ms)	Comparision	Running time (ms)	Comparision	Running time (ms)	Comparision
Selection Sort	8813.13	10000199998	87085.6	90000599998	291427	250000999998
Insertion Sort	1.004	889310	2.51	1330870	4.004	2014918
Shell Sort	5.2354	3080153	17.971	10276591	25.8391	17085419
Bubble Sort	1809.01	1680088130	5925.02	5609639819	55702.8	10788100984
Heap Sort	16.1758	8364782	52.8144	27413229	91.7076	47404999
Merge Sort	12.9264	4896990	38.5546	15778570	64.4168	27198346
Quick Sort	5.3525	3204891	17.3189	10473233	28.466	18119485
Radix Sort	3.3741	1700125	14.6825	6000150	22.7491	10000150
Counting Sort	0.5552	199998	1.58	599998	2.686	999998
Binary Insertion Sort	2.9018	4957538	61.6558	16115480	105.488	27534296
Shaker Sort	16475.3	10000050001	155603	90000150001	462126	250000250001
Flash Sort	4.505	409991	11.686	1229989	21.765	2049991

3.2. Visualizing

3.2.1. Line Graphs

Line Graphs Randomized

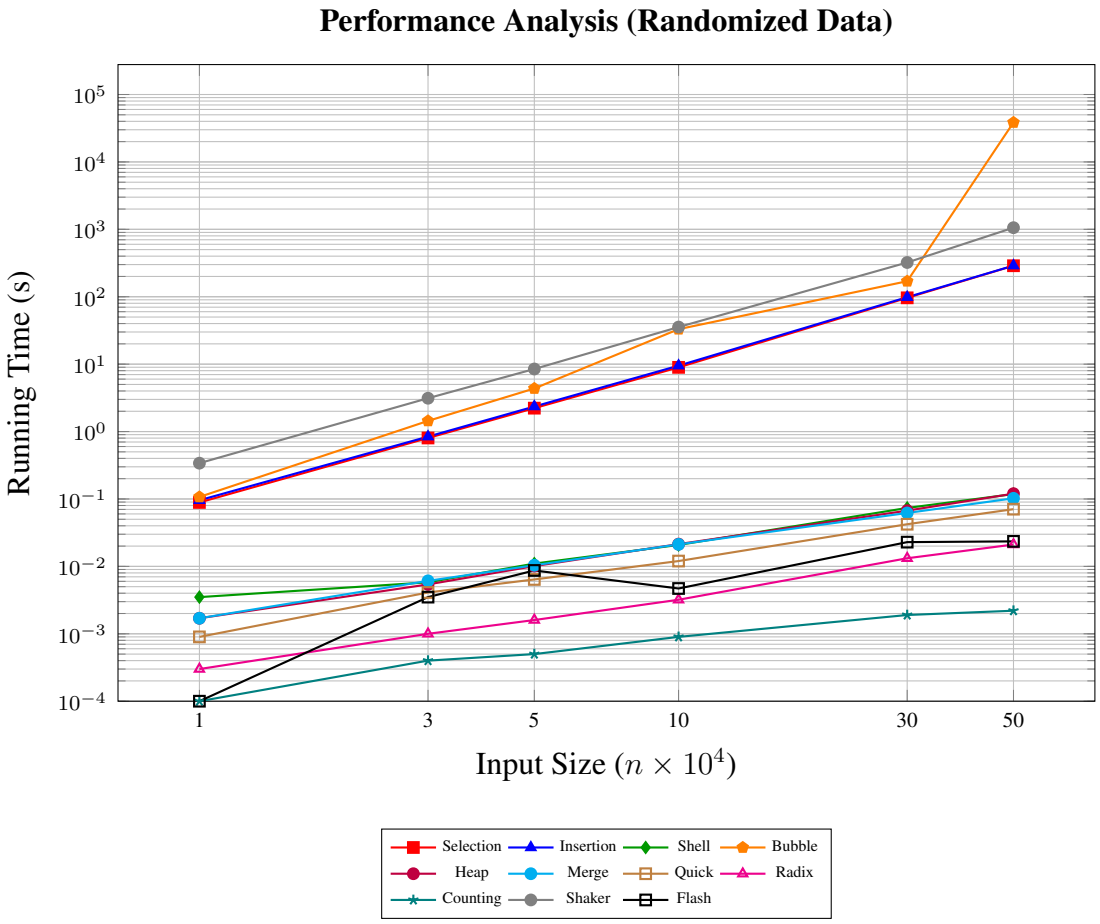


Figure 3.1: Running time performance of algorithms on Randomized Data.
Comments: Key Takeaways:

- **The $O(n^2)$ Explosion:** Bubble, Shaker, Insertion, and Selection Sort show steep, almost vertical growth as n increases. Bubble Sort, in particular, reaches over 38,000 seconds at $500,000$ elements, making it unusable for large datasets.
- **The Efficient Elite:** Quick, Merge, Heap, and Shell Sort remain consistently fast, appearing as near-horizontal lines at the bottom of the log scale. They handle $500,000$ elements in well under 1 second.
- **The Speed Demons:** Counting, Radix, and Flash Sort dominate at all sizes, maintaining exceptionally low execution times due to their non-comparison-based nature.

Line Graphs Sorted

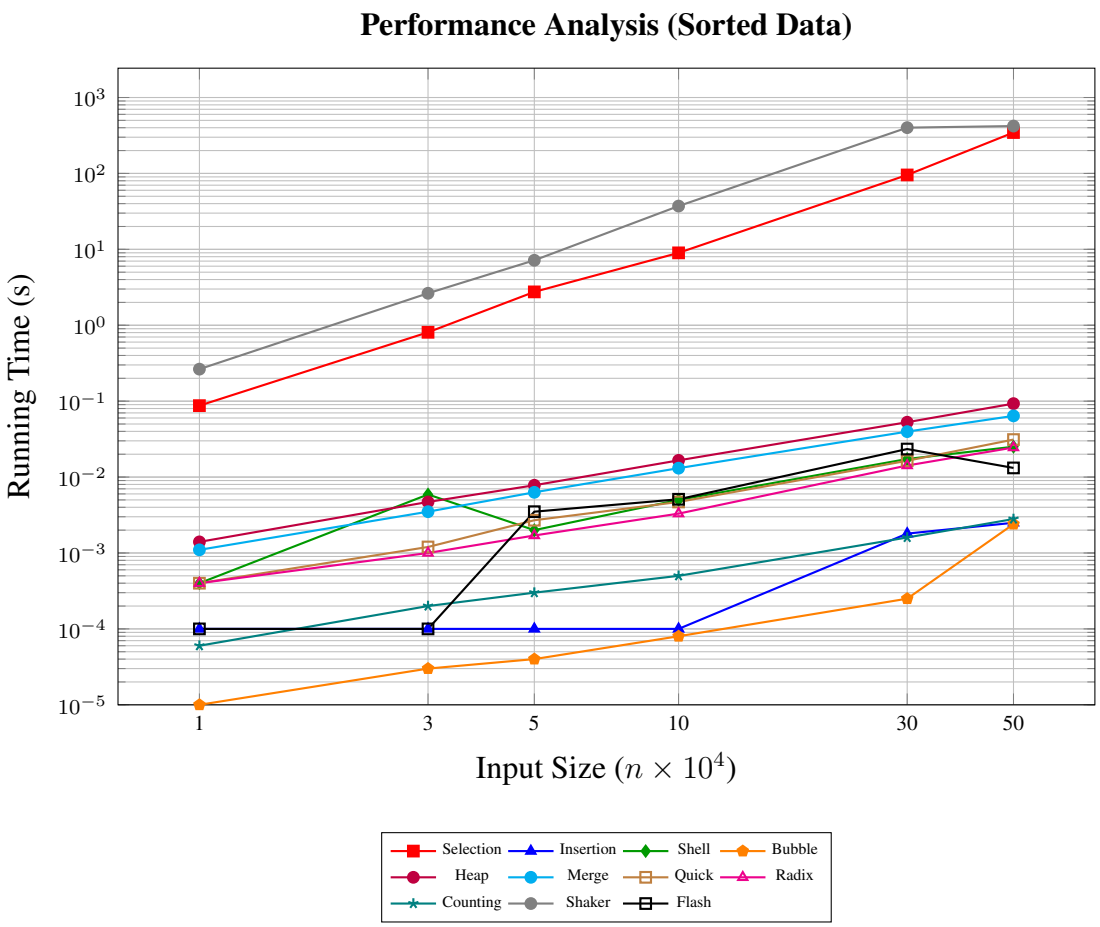


Figure 3.2: Running time performance of algorithms on Sorted Data.

Comments: Key Takeaways:

- **The Adaptive Champions:** Insertion Sort and Bubble Sort transform into $O(n)$ performers. Their lines drop significantly compared to randomized data, as they only need a single pass to confirm the order.
- **The Blind Performers:** Selection Sort remains extremely slow. Its execution time doesn't improve because it always performs the same number of comparisons and swaps regardless of initial order.
- **Consistent Efficiency:** Advanced algorithms like Merge and Heap Sort maintain their reliable performance, showing little change from their randomized data results.

Line Graphs Reversed

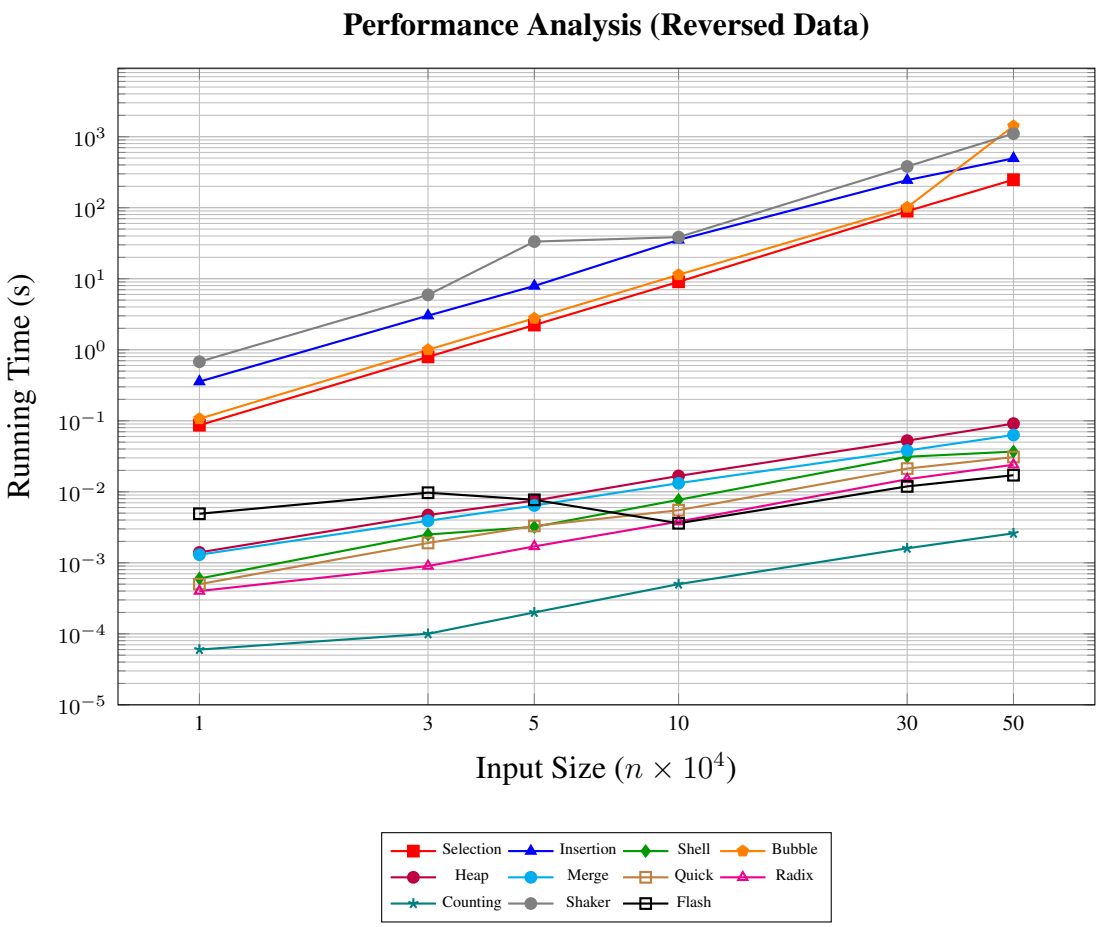


Figure 3.3: Running time performance of algorithms on Reversed Data.

Comments: Key Takeaways:

- **Maximum Penalty:** Standard Insertion Sort and Bubble Sort suffer their worst-case performance, as every single element must be moved across the entire array.
- **Robust Algorithms:** Merge Sort, Heap Sort, and Quick Sort continue to perform efficiently, proving their stability even against the most difficult input arrangements.
- **Shock to Shaker:** Shaker Sort shows a significant performance hit compared to Nearly Sorted data, as the alternating passes become extremely long in a fully reversed scenario.

Line Graphs Nearly Sorted

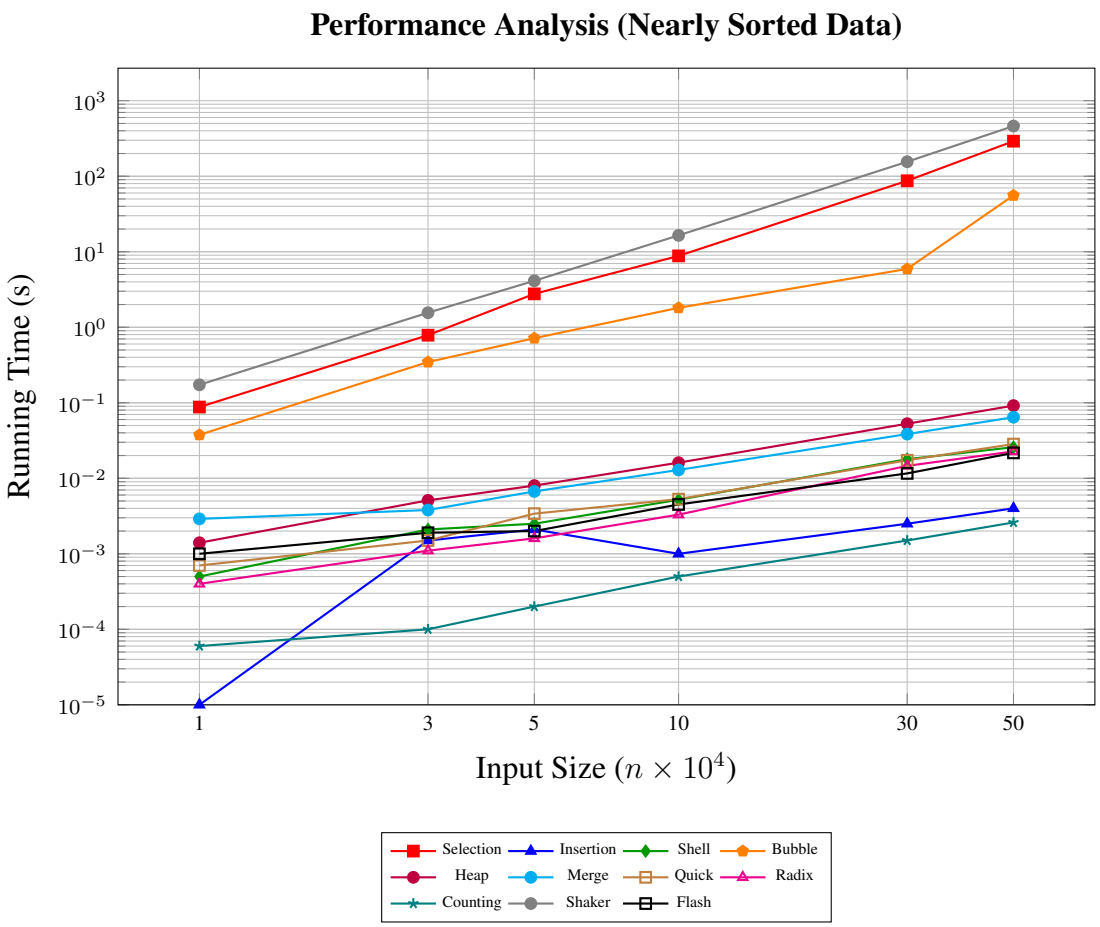


Figure 3.4: Running time performance of algorithms on Nearly Sorted Data.

Comments: Key Takeaways:

- **Near-Instant Success:** Insertion Sort is the clear winner here, performing almost instantly. It is the ideal choice for data that is already close to being sorted.
- **Smooth Sailing:** Most algorithms show improved performance compared to the Reversed or Randomized cases, as the "disorder" that triggers heavy processing is minimal.
- **Initial Order Sensitivity:** Shaker Sort and Bubble Sort are noticeably faster here than in the Randomized case, showing their strong dependency on the initial degree of order.

3.2.2. Bar Charts

Bar Charts Randomized

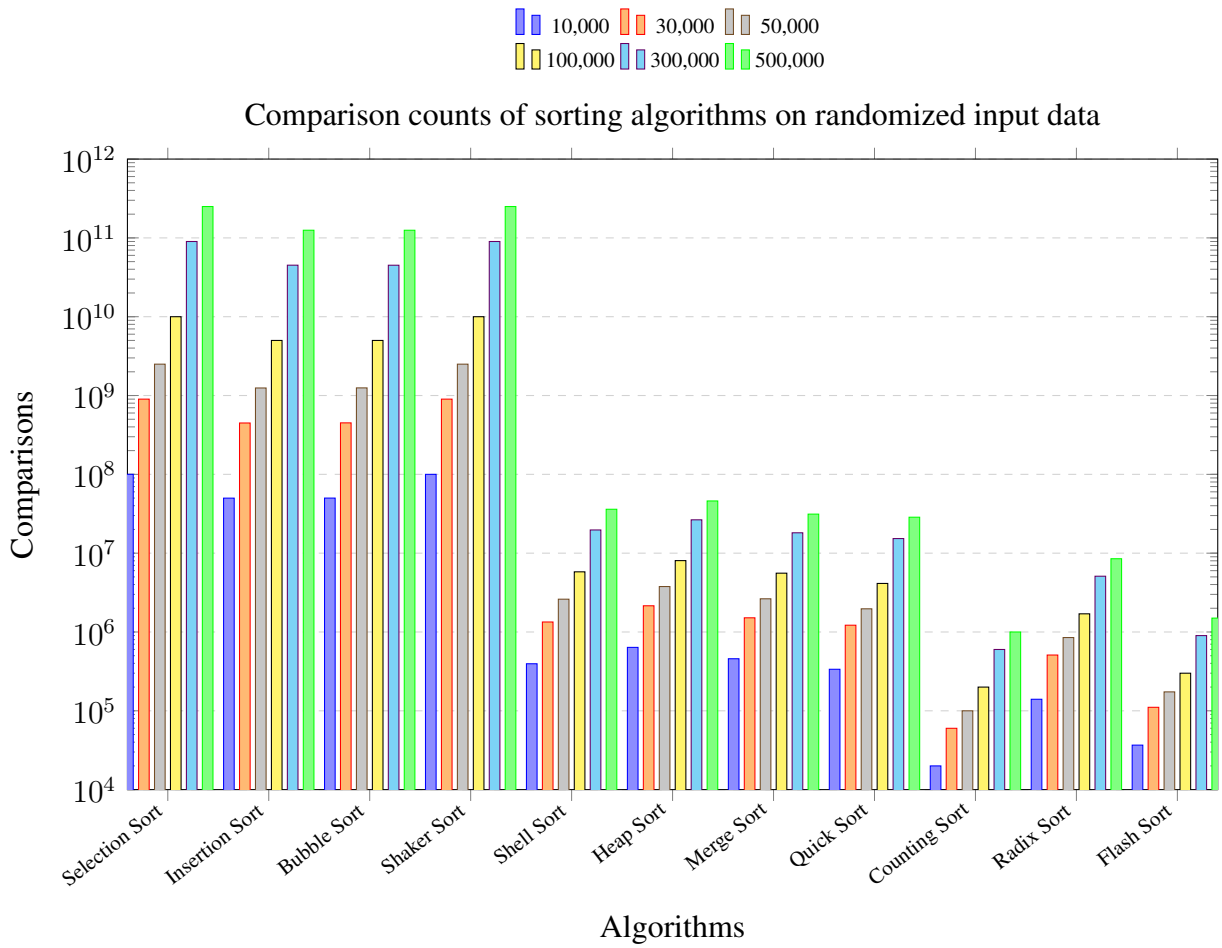


Figure 3.5: Comparison counts of sorting algorithms on randomized input data

Key Takeaways.
Worst performers: Selection Sort, Insertion Sort, Bubble Sort, and Shaker Sort are the least efficient on randomized data. Their bars are the tallest, which means they require a very large number of comparisons as the input size grows.
Balanced performers: Shell Sort, Heap Sort, Merge Sort, and Quick Sort perform much better. Their comparison counts are significantly lower and increase more moderately.
Best performers: Counting Sort, Radix Sort, and Flash Sort have the lowest bars. This indicates that they need far fewer comparisons and remain highly efficient even for large datasets.

Bar Charts Sorted

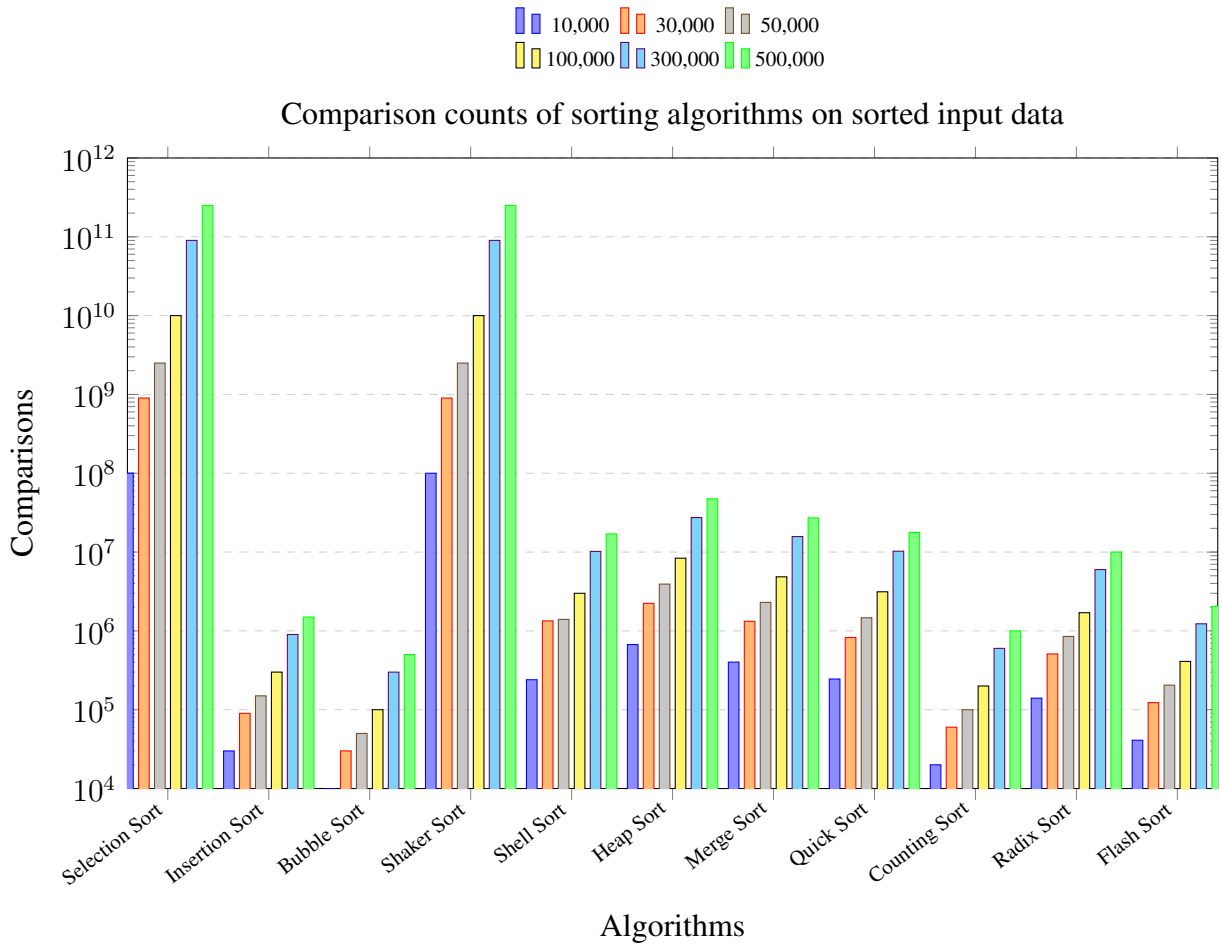


Figure 3.6: Comparison counts of sorting algorithms on sorted input data

Performance on Sorted Data.
When the input data is already sorted, algorithms that can take advantage of this condition perform much better. **Insertion Sort** has very low comparison counts because each element is already in the correct position, so only a small number of checks is needed. **Counting Sort** and **Radix Sort** also remain efficient, since their performance is not heavily affected by the initial order of the data.
In contrast, **Selection Sort** and **Shaker Sort** still require a large number of comparisons even when the array is already sorted. Their bars remain very high and increase significantly as the input size grows, showing that these algorithms do not benefit much from sorted input.
Bubble Sort performs better than Selection Sort and Shaker Sort on sorted data, which suggests that the implementation is able to stop early when no swaps are needed. Meanwhile, algorithms such as **Heap Sort**, **Merge Sort**, **Quick Sort**, and **Shell Sort** show moderate comparison counts, lying between the most efficient and the least efficient groups in this test.

Bar Charts Reversed

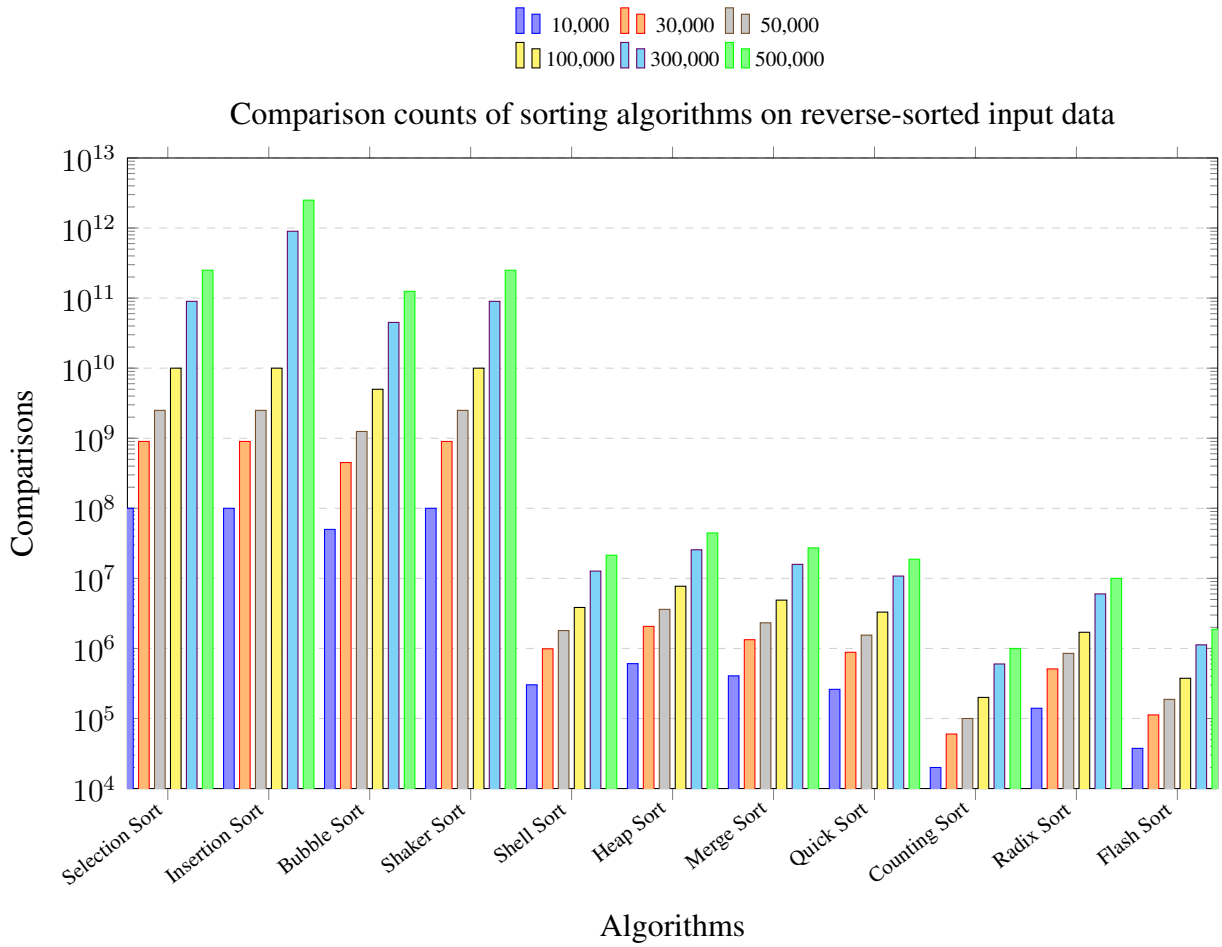


Figure 3.7: Comparison counts of sorting algorithms on reverse-sorted input data

Performance on Reverse-Sorted Data.
The reverse-sorted dataset presents one of the most challenging scenarios for several simple sorting algorithms. In particular, **Insertion Sort**, which performed exceptionally well on already sorted data, experiences a significant decline in efficiency in this case. Because each element is initially positioned as far as possible from its correct location, the algorithm must repeatedly shift elements across a large portion of the array. As a result, the number of comparisons and movements increases dramatically, leading to near worst-case performance.
Meanwhile, **Selection Sort**, **Bubble Sort**, and **Shaker Sort** continue to show relatively weak performance. Their behavior on reverse-sorted input is similar to their performance on randomized data, requiring a large number of comparisons and demonstrating limited adaptability to input order.
In contrast, the more advanced algorithms—including **Merge Sort**, **Quick Sort**, **Counting Sort**, **Radix Sort**, and **Flash Sort**—remain largely unaffected by this unfavorable data arrangement. Their results are nearly identical across sorted, random, and reverse-sorted datasets. This consistency highlights their robustness and efficiency, as their performance depends far less on the initial ordering of the input data.

Bar Charts Nearly Sorted

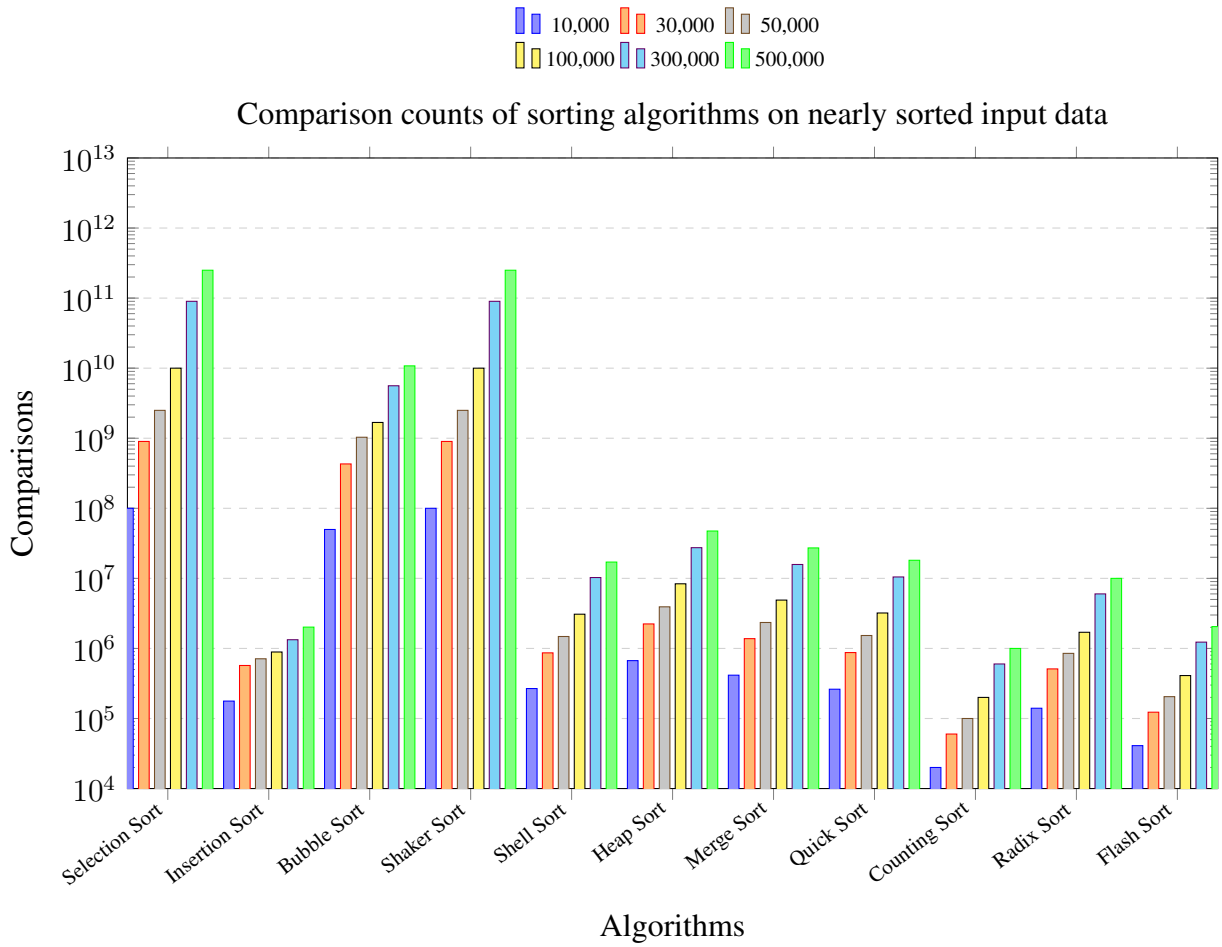


Figure 3.8: Comparison counts of sorting algorithms on nearly sorted input data

Performance on Nearly Sorted Data.
The nearly sorted dataset is a favorable case for algorithms that can take advantage of existing order. In particular, **Insertion Sort** performs much better than it does on randomized or reverse-sorted data because only a limited number of elements are out of place, so fewer shifts are required.
However, **Selection Sort**, **Bubble Sort**, and **Shaker Sort** still show relatively high comparison counts. Their performance improves only slightly because they are not highly adaptive to small amounts of disorder in the input.
By contrast, the more advanced algorithms—including **Shell Sort**, **Heap Sort**, **Merge Sort**, **Quick Sort**, **Counting Sort**, **Radix Sort**, and **Flash Sort**—continue to perform efficiently. Their results remain stable across different input patterns, which demonstrates their robustness and scalability.

3.2.3. Overall Comments

Table 3.1: Comparison Table

Efficiency	**Algorithm**	**Comments**
High Performance	Counting Sort, Radix Sort, Flash Sort, Quick Sort, Merge Sort, Heap Sort, Shell Sort	These algorithms are the most efficient compared to all types of sorting in the charts. Non-comparison sorts (Counting, Radix, Flash) lead the pack, while the others provide excellent $O(n \log n)$ or near-$O(n \log n)$ performance.
Low Performance	Selection Sort, Bubble Sort, Shaker Sort	These are the slowest $(O(n^2))$. As N grows to 500,000, their comparison counts explode, making them unsuitable for large data.
Stable	Merge Sort, Insertion Sort, Bubble Sort, Counting Sort, Radix Sort	They keep the original order of equal elements, which is useful when stability is required.
Unstable	Quick Sort, Heap Sort, Selection Sort, Shell Sort, Shaker Sort, Flash Sort	These algorithms may change the relative order of equal elements.

Binary Insertion Sort is not included in the line graph and bar chart because its overall time complexity remains $O(n^2)$, similar to standard Insertion Sort. Although it uses binary search to reduce the number of comparisons, the dominant cost of shifting elements still results in nearly identical running time. Therefore, including it would not provide significant additional insight and could lead to redundant visualization.

4. Project organization and Programming notes

4.0.1. Project organization

The project is organized in a modular manner so that each component has a clear and simple responsibility.

The main entry point of the program is `main.cpp`. This file is intentionally kept short. Its main role is to receive command-line arguments, call the command parser, and then pass the parsed command to the command runner. This design improves readability and makes the main program easier to maintain.

The command parsing logic is implemented in `CommandHandle.h` and `CommandHandle.cpp`. This module is responsible for reading `argc` and `argv`, validating the command format, and determining which command is being used. The project supports five commands in total. After parsing, the extracted information is stored in a `Command` structure.

The `Command` structure and the related enumerations are defined in `CommandType.h`. This file stores important execution information such as the mode, command type, algorithm names, input file name, input size, input order, and output parameter. Grouping these values into a single structure makes the program more organized, since one object can represent all information needed for a single execution.

The execution logic is implemented in `Command_Runner.h` and `Command_Runner.cpp`. This module performs the main tasks of the program, including handling Commands 1 to 5, reading input files, generating datasets, executing sorting algorithms, measuring running time, counting comparisons, and printing or writing the results.

Helper functions for converting strings into internal indices are placed in `Getdata.h` and `Getdata.cpp`. This module maps algorithm names such as `quick-sort`, `merge-sort`, or `shell-sort` to integer indices, and also maps input orders such as `-rand`, `-sorted`, `-rev`, and `-nsorted`. This reduces repeated string comparisons and makes the execution logic simpler.

String-related utility functions are implemented in `StringHandle.h` and `StringHandle.cpp`. This module checks whether a parameter is numeric, whether an input order is valid, and whether an output parameter is valid. Separating these checks into a dedicated module helps keep the parser concise and avoids duplicated code.

The benchmark logic is implemented in `Command_Runner.cpp`. This module uses `chrono` to measure the running time of sorting algorithms. Two benchmark modes are used: one for timing only and another for timing together with comparison counting. This separation makes the benchmarking code clearer and easier to reuse.

The data generation logic is implemented in `DataGenerator.h` and `DataGenerator.cpp`. This module creates arrays for the required input orders, including random order, sorted order, reverse order, and nearly sorted order. These generated datasets are used in Commands 2, 3, and 5.

Each sorting algorithm is implemented in a separate source file. For example, the project includes files such as `selection_sort.cpp`, `insertionSort.cpp`, `shell_sort.cpp`, `bubble_sort.cpp`, `heap_sort.cpp`, `merge_sort.cpp`, `quick_sort.cpp`, `radixSort.cpp`, `flashSort.cpp`, `shakerSort.cpp`, and `binaryInsertionSort.cpp`, together with their corresponding header files. This structure makes each algorithm independent and easier to test, debug, and modify.

Finally, `sorts.h` serves as a common header that collects the declarations of all sorting functions. This allows other parts of the program to include a single header instead of many separate algorithm headers, making the code cleaner and easier to manage.

4.0.2. Programming notes

The program supports two execution modes: **algorithm mode** and **comparison mode**. Algorithm mode is selected using the parameter `-a`, while comparison mode is selected using `-c`.

In algorithm mode, the program runs a single sorting algorithm on a specific dataset. The dataset can either be read from an input file or generated automatically based on the given input size and data order. In comparison mode, the program runs two different sorting algorithms on the same dataset and compares their performance.

Command 1

```
groupid.exe -a Algorithm Input_file Output_parameter
```

This command reads the input array from the specified file, executes the selected sorting algorithm, and reports the required output according to `-time`, `-comp`, or `-both`.

Command 2

```
groupid.exe -a Algorithm Input_size Input_order Output_parameter
```

This command first generates a dataset with the specified size and order. The generated array is written to `input.txt`. The algorithm is then executed and the sorted result is written to `output.txt`.

Command 3

```
groupid.exe -a Algorithm Input_size Output_parameter
```

This command generates four different dataset types: random, nearly sorted, sorted, and reverse sorted data. The generated arrays are written to `input_1.txt`, `input_2.txt`, `input_3.txt`, and `input_4.txt`. The selected algorithm is then executed on all four datasets.

Command 4

```
groupid.exe -c Algorithm_1 Algorithm_2 Input_file
```

This command compares two algorithms using the same input file. Since both algorithms must run on identical data, the original array is copied into two separate arrays before execution.

Command 5

```
groupid.exe -c Algorithm_1 Algorithm_2 Input_size Input_order
```

This command first generates one dataset with the specified size and order. The dataset is then copied into two arrays so that both algorithms run on exactly the same input, ensuring a fair comparison.

Two performance metrics are measured in this project: running time and the number of comparisons. When the option `-both` is used, the program does not reuse the same sorted array for both measurements. Instead, two separate copies of the input array are created. One copy is used for measuring execution time, while the other copy is used for counting comparisons. This avoids incorrect comparison counts that may occur if the algorithm is executed on an already sorted array.

Dynamic arrays are used in this project instead of `vector`, according to the project requirements. Therefore, memory management must be handled carefully. Every array created with `new[]` must be released using `delete[]` after use in order to avoid memory leaks.

Before execution, the program validates all command-line parameters. This includes checking whether the execution mode is valid, whether the algorithm name exists, whether the input size is numeric, whether the input order is valid, and whether the output parameter is valid. Early validation helps prevent invalid execution and makes debugging easier.

During implementation, several common issues were identified and corrected. One example was mistakenly using the same array for both time measurement and comparison counting. Another issue occurred when attempting to use the output parameter in comparison commands, even though Commands 4 and 5 do not require this parameter. These problems were resolved by separating the execution logic more clearly and using dedicated helper functions.

It is also important to note that running time measurements for very small input sizes may not be stable. For example, when the input size is only 10 or 20, the measured execution time may vary slightly between runs because the values are extremely small. Therefore, timing results are more meaningful for medium or large datasets, while small datasets are mainly used to verify correctness.

Overall, the project is designed with a modular structure that separates command parsing, data generation, execution, benchmarking, and sorting logic. This design improves readability and makes the program easier to test, debug, maintain, and extend.

5. References

Websites, textbooks, and articles used in the report:

- GeeksforGeeks, *Can We Improve the Performance of a Merge Sort Algorithm?*
  https://www.geeksforgeeks.org/dsa/can-we-improve-the-performance-
- Wikipedia, *Shellsort*
  https://en.wikipedia.org/wiki/Shellsort#:~:text=It%20can%20be%20understood%20as,between%20elements%20to%20be%20compared.
- GeeksforGeeks, *Shell Sort*
  https://www.geeksforgeeks.org/dsa/shell-sort/
- W3Schools, *Merge Sort*
  https://www.w3schools.com/dsa/dsa_algo_mergesort.php
- GeeksforGeeks, *Quick Sort Algorithm*
  https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/
- GeeksforGeeks, *Insertion Sort Algorithm*
  https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/
- YouTube, *Binary Insertion Sort*
  https://www.youtube.com/watch?v=XsKZTCzLhdo
- GeeksforGeeks, *Binary Insertion Sort*
  https://www.geeksforgeeks.org/dsa/binary-insertion-sort/
- Sort Visualizer, *Shaker Sort*
  https://sortvisualizer.com/shakersort/
- AlgoLab, *Flash Sort*
  http://algolab.valemak.com/flash
- YouTube, *Flash Sort Why O(n) Time complexity?*
  https://www.youtube.com/watch?v=CAaDJJUszvE
- GeeksforGeeks, *C++ Program For Radix Sort*
  https://www.geeksforgeeks.org/cpp-program-for-radix-sort/
- YouTube, *Sorting Algorithm Video Reference*
  https://youtu.be/7h1s2SojIRw?si=r8uVWgGwq97QQd2H
- GeeksforGeeks, *Heap Sort*
  https://www.geeksforgeeks.org/dsa/heap-sort/
- Stack Overflow, *Optimize Counting Sort*
  https://stackoverflow.com/questions/36107004/optimize-counting-sor
- GeeksforGeeks, *Counting Sort*
  https://www.geeksforgeeks.org/dsa/counting-sort/

- GeeksforGeeks, *Compiling With g++*
  https://www.geeksforgeeks.org/linux-unix/compiling-with-g-plus-plu
- Stack Overflow, *Optimized Selection Sort*
  https://stackoverflow.com/questions/35643105/optimized-selection-s
- GitHub, *HCMUS Sorting Algorithms Repository*
  https://github.com/nqthangcs/HCMUS-sorting-algorithms

6. Acknowledgements

We would like to express our sincere gratitude to everyone who supported us during the completion of this project.

We would like to thank our instructors for providing valuable knowledge and guidance throughout the Data Structures and Algorithms course. Their lectures and materials helped us understand the fundamental concepts required for this project.

We also appreciate the support and collaboration among team members, which made the implementation and analysis process smoother.

We would also like to thank Mr.Hoang Quan Nguyen for allowing us to refer to his previous project. His work was a helpful reference during the development of this assignment.

Special thanks to Nha Van Viet Code for providing optimizations for Bubble Sort.

Finally, we would like to acknowledge the assistance of modern tools, including artificial intelligence systems such as ChatGPT,Gemini , which helped us review LaTeX formatting, improve the clarity of explanations, and refine the structure of the report, rewrite the comment in formal form.